

Part - 1 : Linear Regression

Introduction

- These ^{are} methods that learn a model that is a continuous fn. of its ~~an~~ inputs.
- This class of methods are specially useful when we want to predict an outcome, along with the relationship b/w the input variables & the outcome.
- This relationship can be used as advice on how to get the outcome that you want to achieve.

* Using Linear Regression

If we are trying to predict a numerical quantity like profit, cost or sales volume first we should try linear regression.

→ [Here we use linear regression for a real-world example of predicting personal income]

→ Regression analysis is a statistical process for estimating relationships b/w variables.

→ It includes many techniques for modelling & analyzing several variables when the focus is on the relationship b/w a dependent variable (target or response variable) & one or more independent variables (called predictors)

* Simple linear regression is used to determine the extent of the linear ~~regression~~ relationship b/w a dependent variable & a single independent variable.

* Regression Analysis is used for the following purposes

- i) prediction of the target variable (forecasting)
- ii) modelling the relationship b/w x & y and
- iii) testing of hypotheses.

⇒ Model fitting

Models are a representation of a sequence of data points, ~~which has~~

→ Model fitting refers to choosing the right model that best describes a set of data.

→ R has different types of models:

- Linear model (lm): `lm()` is a linear model fn: in R. It can be used to create a simple ~~&~~ regression model.
- Generalised linear model (glm): It is specified by giving a symbolic description of linear predictor & a description of error distribution.
- Linear model of mixed effects (lme)
- Non-linear least squares (nls): It determines the non-linear (weighted) least square estimate of the parameters of a non-linear model.
- Generalised ~~additive~~ additive models (GAM):

GAM's are simply a class of statistical models in which the usual linear relationship b/w the response & predictors is replaced by several ~~nonlinear~~ nonlinear smooth fns.

LINEAR REGRESSION

- Linear regression in R consists of two main variables that are related through an equation where exponent of both variables is 1.
- Mathematically linear relationship represents a straight line when plotted as graph.
- In non linear relationship the exponent of the variables is not equal to 1 & it creates a curve on the graph.
- General equation of ~~non~~ linear regression is $y = ax + b$. where y is a response variable, x is a predictor variable & a & b are constants called coefficients.

⇒ lm() fn: in R

* $\text{lm}(\text{formula}, \text{data}, \text{subset}, \text{weights}, \text{na.action}, \text{method} = "qr", \text{model} = \text{TRUE}, \text{x} = \text{FALSE}, \text{y} = "FALSE", \text{qr} = \text{TRUE}, \text{singular.ok} = \text{TRUE}, \text{contrasts} = \text{NULL}, \text{offset}, \dots)$

where

- formula - represents relation b/w x & y ,
- data - contains the variable in the model,

- subset - optional vector that specifies a subset of observations used in model fitting.
- weights - optional vector that specifies the weight for the model fitting process.

It takes a numeric vector value or NULL.

- na.action - optional fn: that specifies the actions on how to react for data that contains NAs.
- method - method used in fitting.
- model, x, y, qr - If this parameter is TRUE, then model matrix, model frame & QR decomposition are returned.
- singular.ok - If this parameter is FALSE, then a singular fit is an error.
- Contrast - is an optional list object used to specify prior known components that are to be included in the linear predictor.

⇒ Simple syntax of `lm()` fn: is linear regression
is `lm(formula, data)` where optional parameters can be omitted

⇒ Here will determine the relationship b/w the predictor & response variables for a student data set.

· Freshmen Score - response variable (y)

No. of hrs - predictor variable (x)

$$y = ax + b \quad a, b \text{ - constants}$$

- Use the `cor()` function in R to determine the Degree & Direction of Linear Association.
- The degree & direction of a linear association can be determined using correlation.

Step 1 Predictor vector $x \rightarrow$ heights of persons
 1) $x \leftarrow c(152, 172, 139, 187, 129, 137, 180, 162, 151, 130)$

Step 2
 2) $y \leftarrow c(62, 80, 55, 90, 48, 56, 75, 73, 63, 49)$

response vector. $y \rightarrow$ wts of persons.

Step 3:

Apply $lm()$ fn:
 $\rightarrow relation \leftarrow lm(y \sim x)$
 $print(relation)$

Demonstrate the relationship model
 b/w predictor & response variables.

Call:

$lm(formula = y \sim x)$

coefficients

(Intercept) x

-34.7196 0.6473

Step 4: print the summary of the relationship.

$\rightarrow print(summary(relation))$

Step 5: find the wt of a person with height 170

$a \leftarrow data.frame(x = 170)$

$result \leftarrow predict(relation, a)$

$print(result)$

6) visualize the regression graphically by plotting a chart

```
> plot (y, x, col = "blue", main = "Height & wt  
regression",  
+ abline(lm(x ~ y)), cex = 1.3, pch = 16,  
xlab = "wt in kg", ylab = "Ht in cm")
```

Unsupervised Methods

①

- unsupervised methods discover unknown relationships in data. with these methods there is no outcome to predict, instead we discover patterns in the data that ~~per~~ were not suspected previously.
- Here we find the patterns that build the predictive models
- * Here we have two classes of unsupervised methods
 - cluster analysis - finds groups in data with similar characteristics.
 - Association rule mining - finds elements or properties in the data that tend to occur together.

Cluster Analysis

- The goal is to group the observations ~~into~~ in our data into clusters such that every data point in a cluster is more similar to other datapoints in the same cluster & is different from other clusters.

ex guided tours that a company offers; that might want to cluster its clients by behaviour & tastes, which countries they want to visit, whether they prefer adventure tours, luxury tours, or educational tours, what kinds of activities they want to participate in etc...

* Such information can help the company design attractive travel packages & target appropriate segments of clients.

→ ⁱⁿ cluster analysis we discuss two approaches:

i) Hierarchical clustering: finds nested groups of clusters.

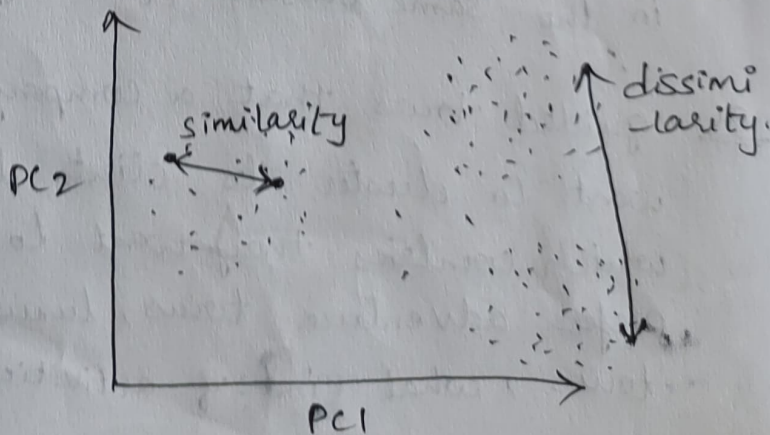
Ex plant taxonomy - which classifies ~~plants~~ plants by family, genus, species & so on...

ii) K-means clustering: A quick & popular way of finding clusters in ^{quantitative} ~~quantitative~~ data.

* Distances

In order to cluster we need to notations of similarity & Dissimilarity.

→ Dissimilarity is a distance, in which the datapoints in a cluster are far away from the data points in other clusters.



* Different applications areas have different notations of distance & dissimilarity.

Some of them are :-

- Euclidean distance
- Hamming distance
- Manhattan distance
- Cosine similarity.

i) Euclidean Distance

- The most common distance is Euclidean distance.
- The euclidean distance b/w two vectors x & y is defined as

$$\text{edist}(x, y) \leftarrow \sqrt{((x[1] - y[1])^2 + (x[2] - y[2])^2 + \dots)}$$

- This optimised square euclidean distance is the basis of k-means.
- euclidean distance is only used when all the data is real-valued (quantitative)
- If data is categorical, other distances can be used.

Ex : find euclidean distance b/w points

$$P(3,2) \text{ \& } Q(4,1)$$

$$P(3,2) = (x_1, y_1)$$

$$Q(4,1) = (x_2, y_2)$$

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$= \sqrt{(4-3)^2 + (1-2)^2}$$

$$= \sqrt{1^2 + 1^2} = \sqrt{2}$$

euclidean distance b/w P & Q is $\sqrt{2}$.

i) Hamming Distance

For categorical variables (male/female, small/medium/large) we can define the distance as
0 if two points are in same category &
1 otherwise.

→ If variables are categorical we can use Hamming distance, which counts the no. of mismatches

$$\text{hdist}(x, y) \leftarrow \text{sum} ((x[1] \neq y[1]) + (x[2] \neq y[2]) + \dots)$$

Here $a \neq b$ is defined to have a value of 1, if the expression is true & a value of 0, if expression is false.

→ If the categories are ordered (like S/M/L)
i.e. the categories are closer, then we can convert
them into a numerical sequence.

i.e. $(S/M/L) \Leftrightarrow (1/2/3)$, Then we can use
Euclidean distance or other distance metrics for
quantitative data.

iii) Manhattan Distance ^(city block): Measures distance in the no:
of horizontal & vertical units it takes to get
from one (real-valued) point to the other (No diagonal
moves)

$$\text{mdist}(x, y) \leftarrow \text{sum}(\text{abs}(x[1] - y[1]) + \text{abs}(x[2] - y[2]))$$

Ex

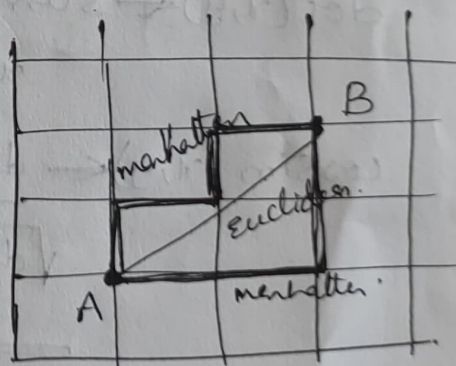
$$I/P \quad n = 4$$

$$P_1 = \{-1, 5\}$$

$$P_2 = \{1, 6\}$$

$$P_3 = \{3, 5\}$$

$$P_4 = \{2, 3\}$$



Distance of P_2, P_3, P_4 from P_1

$$\text{are } \text{sum}((|1+1| + |6-5|), (|3+1| + |5-5|), (|2+1| + |3-5|))$$

$$= \text{sum}(\underset{3}{(2+1)}, \underset{4}{(4+0)}, \underset{5}{(3+2)})$$

$$= 12$$

→ ||y Distance of P_3, P_4 from P_2

are 3, 4 resp

$$\therefore \text{sum} = 12 + 3 + 4 = 19$$

||y Distance of P_4 from P_3 is 3

$$\therefore \text{sum} = 19 + 3 = 22$$

iv) Cosine Similarity : It is a common similarity metric in text analysis.

→ It measures the smallest angle b/w two vectors.

→ Two perpendicular vectors are most dissimilar as

$$\cos(90^\circ) = 0. \quad \cos(0^\circ) = 1$$

→ Two parallel vectors are most similar. (Identical)

Ex: $x = \{3, 2, 0, 5\}$
 $y = \{1, 0, 0, 0\}$

$$\text{dot}(x, y) \leftarrow \text{sum}(x[1]*y[1] + x[2]*y[2] + \dots)$$

$$x \cdot y = 3*1 + 2*0 + 0*0 + 5*0 = 3$$

$$\text{cosSim}(x, y) \leftarrow \frac{\text{dot}(x, y)}{\sqrt{\text{dot}(x, x) * \text{dot}(y, y)}}$$

$$|x| = \sqrt{3^2 + 2^2 + 0^2 + 5^2} = 6.16$$

$$|y| = \sqrt{1^2 + 0^2 + 0^2 + 0^2} = 1$$

cosine similarity

$$S_c = \frac{3}{(6.16 \times 1)} = 0.49$$

(smaller the angle, higher the similarity) $\therefore D_c = 1 - 0.49 = 0.51$

The cosine similarity b/w two vectors is measured in θ .

If $\theta = 0^\circ$ Then x & y vectors overlap i.e. Similar

if $\theta = 90^\circ$ " " " are dissimilar.

Preparing the Data

* Hierarchical clustering

- Clustering is the most common form of unsupervised learning, a type of mlc learning algorithm used to draw interfaces from unlabelled data.
- clustering algorithms group a set of data points into subsets or clusters.
- The main goal is entities within a cluster should be as similar as possible & entities in one cluster should be different from entities in another cluster.

* Broadly speaking there are two ways of clustering data points based on the algorithmic structure & operation :-

- i) Agglomerative : An agglomerative approach begins with each observation in a distinct cluster, & successively merges clusters together until a stopping criterion is satisfied.
- ii) Divisive : A divisive method begins with all patterns in a single cluster & performs splitting until a stopping criterion is met.

- Here we focus on agglomerative or bottom-up approach, where we start with each data point as its own cluster & then combine clusters based on some similarity measure.
- This similarity measure b/w the clusters is often calculated using different distance metrics like euclidean distance b/w two clusters.
- This distance measures depends on the type of data in the dataset.

* Preparing Data (pre-processing)

To begin with clustering process we need to scale our data, because each observations feature values are represented as coordinates in n -dimensional space (n is no. of features)

- & then the distances b/w these co-ordinates are calculated.

for ex: ht & wt of 3 people: A (6ft, 75kg)

B (6ft, 77kg)

C (8ft, 75kg)

so if we represent these features in a 2D coordinate system, & cal the euclidean

dist b/w them, then A-B : 2 units

A-C : 2 units

here it shows that both the pairs A-B & A-C are similar, but they are not.

→ The pair A-B is more similar than pair A-C so its imp. to scale these values first & then cal. the distance.

→ There are various ways to normalize the feature values, min-max standardization, standard deviation, we can use $\text{normalize}()$ fn: in R for this purpose. This can be achieved via $\text{scale}()$ fn:

→ Also we need to handle missing / null / inf values in the dataset ~~in preprocessing~~

* Hierarchical clustering Algorithm

The key operation here is to repeatedly combine the two nearest clusters into a large cluster.

- i) It starts by calculating the distance b/w every pair of observation points & store it in a distance matrix
- ii) It then puts every point in its own cluster
- iii) Then it starts merging the closest pairs of points

based on the distances from the distance matrix E as a result the amount of clusters goes down by 1.

- iv) Then it recomputes the distance b/w the new cluster E and the old ones E and stores them in a new distance matrix
- v) Lastly it repeats steps 2 & 3 until all the clusters are merged into one single cluster.

* There are several ways to measure the distance b/w clusters, they are often called Linkage methods.

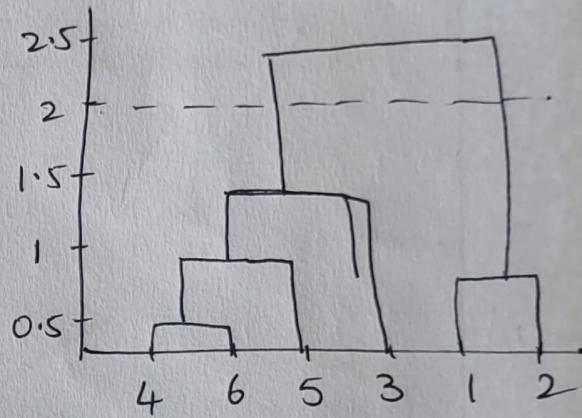
They are: complete linkage - cal's the max. distance b/w clusters before merging.

- single linkage - cal's the min. distance b/w the clusters ~~before~~ before merging.
- Average linkage: calculates the avg distance b/w clusters before merging.
- centroid linkage: finds centroid ~~of~~ cluster 1 & centroid of cluster 2, & then cal's the distance b/w the two before merging.

* Dendrograms

In hierarchical clustering, the objects into a hierarchy similar to a tree like diagram called a dendrogram. The distance of split or merge (called height) is shown on the y-axis.

`kclust()`



* K-Means Clustering

unsupervised non-linear alg that cluster data based on similarity or similar groups.

Hierarchical Clustering in R

Hierarchical clustering is a popular method for grouping data into clusters based on their similarity or dissimilarity. In R, you can perform hierarchical clustering using the `hclust` function and visualize the results with dendrograms. Here's a step-by-step guide on how to perform hierarchical clustering in R:

1. Prepare Your Data:

Ensure that your data is in a suitable format. Hierarchical clustering works best with numeric data, and you may need to normalize or standardize your variables if they have different scales.

2. Calculate the Distance Matrix:

Use a distance metric to calculate the dissimilarity between data points. Common distance metrics include Euclidean distance, Manhattan distance, or correlation distance. The `dist` function in R can be used for this purpose.

R

```
# Example distance calculation (using Euclidean distance)
```

```
distance_matrix <- dist(your_data)
```

3. Perform Hierarchical Clustering:

Use the `hclust` function to perform hierarchical clustering on your data based on the distance matrix.

R

```
# Perform hierarchical clustering (using complete linkage)
```

```
hierarchical_clusters <- hclust(distance_matrix, method = "complete")
```

You can specify different linkage methods, such as "complete," "single," or "average," to control how clusters are formed based on the dissimilarity between points.

4. Plot the Dendrogram:

Visualize the hierarchical clustering results using a dendrogram. The `plot` function is used for this purpose.

R

```
# Plot the dendrogram
```

```
plot(hierarchical_clusters, main = "Hierarchical Clustering Dendrogram", xlab = "Data Points")
```

The dendrogram will show the hierarchy of clusters, with the branches representing the merging of clusters.

5. Cut the Dendrogram to Form Clusters:

Use the `kmeans` function to perform K-means clustering on your data. You need to specify the number of clusters (centers) you want to create.

R

```
# Perform K-means clustering with 3 clusters (you can change the number as needed)
```

```
kmeans_result <- kmeans(data_matrix, centers = 3)
```

5. Explore K-means Results:

After clustering, you can examine the results. The `kmeans_result` object contains several useful pieces of information, including the cluster assignments and cluster centroids.

R

```
# Cluster assignments for each data point
```

```
cluster_assignments <- kmeans_result$cluster
```

```
# Cluster centroids (center of each cluster)
```

```
cluster_centroids <- kmeans_result$centers
```

6. Visualize the Clusters:

You can create visualizations to see how the data points are grouped into clusters. For example, if your data has two numeric variables (e.g., "x" and "y"), you can create a scatterplot:

R

```
# Visualize the clusters in a scatterplot
```

```
plot(your_data$x, your_data$y, col = cluster_assignments, pch = 19,
```

```
main = "K-means Clustering", xlab = "X-axis", ylab = "Y-axis")
```

```
# Add cluster centroids to the plot
```

```
points(cluster_centroids[, 1], cluster_centroids[, 2], col = 1:3, pch = 3)
```

7. Determine the Optimal Number of Clusters:

Selecting the optimal number of clusters is an important step in K-means clustering. You can use methods like the "elbow method" or the "silhouette method" to find the most suitable number of clusters for your data.

8. Interpret and Use the Clusters:

Once you've determined the number of clusters and performed K-means clustering, you can use the cluster assignments for various purposes, such as segmenting your data or making predictions.

K-means clustering is a versatile technique for data partitioning and is widely used for various applications, including customer segmentation, image analysis, and more.

To create distinct clusters, you can cut the dendrogram at a specific height using the `cutree` function. This height represents the desired number of clusters.

R

```
# Cut the dendrogram to form clusters (e.g., 3 clusters)
```

```
clusters <- cutree(hierarchical_clusters, k = 3)
```

You can replace `k` with your desired number of clusters.

6. Visualize the Clusters:

Use the cluster assignments to visualize the data points and their associated clusters.

R

```
# Visualize the clusters
```

```
plot(your_data, col = clusters, main = "Data Points with Cluster Assignments")
```

This code will create a scatterplot with each data point color-coded based on its cluster assignment.

Hierarchical clustering is a versatile technique that allows you to explore the structure of your data and identify meaningful groups. You can adjust the linkage method, distance metric, and number of clusters to suit your specific analysis needs.

K-Means Clustering in R

K-means clustering is a widely used method for partitioning data into clusters based on similarity. In R, you can perform K-means clustering using the `kmeans` function. Here's a step-by-step guide on how to perform K-means clustering in R:

1. Prepare Your Data:

Ensure that your data is in a suitable format. K-means clustering typically works with numeric data. If your data contains categorical variables, you may need to convert them into numerical form or apply other techniques like one-hot encoding.

2. Load Required Libraries:

Start by loading any necessary libraries. In this case, we'll use the base R functions for K-means, so you don't need to install any additional packages.

3. Create a Data Matrix:

If your data is not already in matrix form, create a data matrix with the variables you want to cluster. For example, if you have a data frame called `your_data`, you can create a data matrix like this:

R

```
data_matrix <- as.matrix(your_data)
```

4. Perform K-means Clustering:

* K-Means clustering

- K-means clustering is an unsupervised alg. that cluster data based on similarity or similar groups.
- It partitions the data into a pre-specified no: of clusters. each data point or training example is assigned to a segment called cluster.

* Algorithm:

1. Choose the no: of clusters K .
2. select K points (~~clusters~~ ^{at random} of size 1) at random as centroids.
3. calculate the distance b/w each point & the centroid & assign each data point to the closest cluster.
4. calculate the ^{new} centroid (mean position) for each cluster.
5. Keep repeating steps 3, 4 until the cluster's don't change or the maximum no: of iterations are reached.

* clusterR → package contains centroid based clustering algorithms.

cluster → Inbuilt package

kmeans() → for K-means alg

hclust() → for hierarchical clustering alg.

* Clustering in R

1. Load Data → using fn's like `read.csv()`
2. pre-processed Data - cleaning & preprocessing if needed.
3. choose clustering method - like k-means, hierarchical
4. Feature scaling - Normalize or Standardize data for alg's.
5. Apply clustering Alg → use fn's like `kmeans()` (or `hclust()`)
6. visualize results - plot clusters using `plot()`
or visualize dendrograms.
7. Evaluate clusters → Assess the quality of clusters using ^{diff.} metrics. ~~the~~
8. Interpret Results → ~~do~~ Analyze clusters to gain insight about patterns & relationships with the data.

— . —